

NexusRTOS

A Complete Learning Guide

ARM Cortex-M3 | C + Assembly | QEMU

Build a real-time operating system kernel from zero.

No hardware needed -- runs entirely on QEMU.

Table of Contents

1. What is an RTOS and Why Build One?
2. ARM Cortex-M3 Architecture Crash Course
3. Toolchain Setup (compiler, QEMU, GDB)
4. Project Structure Walkthrough
5. Startup Code & Vector Table
6. The Linker Script
7. Context Switching Deep Dive
8. The Scheduler
9. Task Management
10. SysTick Timer & System Clock
11. Mutexes & Priority Inheritance
12. Semaphores
13. Message Queues (IPC)
14. Heap Allocator
15. UART Driver & printf
16. Debug Shell
17. Building & Running
18. Debugging with GDB
19. KTrace: Kernel Event Tracer
20. Fault Isolation & Auto-Recovery
21. Per-Task CPU Profiler
22. Extending the Project
23. Interview Questions & Answers

1. What is an RTOS and Why Build One?

RTOS vs General-Purpose OS

A Real-Time Operating System (RTOS) is designed to run tasks with deterministic timing guarantees. Unlike Linux or Windows, which optimize for throughput and fairness, an RTOS guarantees that high-priority tasks run within a bounded time (their deadline).

Key differences:

- Deterministic scheduling: worst-case response time is bounded.
- Small footprint: typically kilobytes of RAM and flash.
- No virtual memory / MMU: tasks share a flat address space.
- Cooperative or preemptive multitasking on a single core.

Common RTOS examples: FreeRTOS, Zephyr, ThreadX, VxWorks, QNX.

Why Build Your Own?

Most embedded engineers *use* an RTOS but few understand the internals. Building one from scratch teaches you:

1. How context switching really works at the register level.
2. How schedulers make priority decisions in bounded time.
3. How synchronization primitives prevent race conditions.
4. How memory is managed without malloc/free from a C library.
5. How hardware peripherals (timers, UARTs) interact with the kernel.

This knowledge makes you dramatically more effective at debugging real embedded systems, and it stands out on a resume because almost nobody does it.

2. ARM Cortex-M3 Architecture

Why Cortex-M3?

The Cortex-M3 is the workhorse of the embedded world. It is a 32-bit ARM processor designed for microcontrollers. We target the LM3S6965 (Texas Instruments Stellaris) because QEMU emulates it perfectly, so you need zero hardware.

Key specs of our target:

- CPU: ARM Cortex-M3, 50 MHz
- Flash: 256 KB at address 0x00000000
- SRAM: 64 KB at address 0x20000000
- UART0 at 0x4000C000

Operating Modes

Cortex-M3 has two operating modes:

Thread Mode: normal code execution. This is where your tasks run.

Handler Mode: entered automatically when an exception/interrupt fires.

And two stack pointers:

MSP (Main Stack Pointer): used by exception handlers and at boot.

PSP (Process Stack Pointer): used by tasks in our RTOS.

The CONTROL register bit 1 (SPSEL) selects which stack pointer thread mode uses. Our RTOS switches to PSP for tasks, keeping MSP for the kernel/ISRs. This separation is crucial for safety.

Exception Model

When an exception fires, the hardware automatically pushes 8 registers onto the current stack:

R0, R1, R2, R3, R12, LR, PC, xPSR

This is called the 'exception frame'. The remaining registers (R4-R11) are callee-saved and must be preserved by software -- this is exactly what our PendSV context-switch handler does.

Key exceptions for our RTOS:

- SysTick (exception #15): fires every 1 ms to drive the scheduler.
- PendSV (exception #14): triggered by software for context switches.
We set it to the lowest priority so it runs after all other ISRs.
- SVC (exception #11): supervisor call, used for protected OS calls.

The Vector Table

At address 0x00000000, the processor expects a table of 32-bit values:

Offset 0x00: Initial Stack Pointer (MSP)

Offset 0x04: Reset Handler address

Offset 0x08: NMI Handler

Offset 0x0C: HardFault Handler

... and so on for each exception and interrupt.

On reset, the CPU loads MSP from offset 0 and branches to the address at offset 4 (Reset_Handler). Our `startup.c` defines this table in the `.isr_vector` section, which the linker script places at address 0.

3. Toolchain Setup

ARM Cross-Compiler

We use the GNU ARM Embedded Toolchain (arm-none-eabi-gcc). The 'none' means no operating system -- we ARE the operating system.

Install options:

- Windows: MSYS2 (pacman -S mingw-w64-x86_64-arm-none-eabi-gcc) or download from developer.arm.com
- Ubuntu/WSL: `sudo apt install gcc-arm-none-eabi`
- macOS: `brew install arm-none-eabi-gcc`

Verify: `arm-none-eabi-gcc --version`

QEMU -- The ARM Emulator

QEMU emulates the entire LM3S6965 board: CPU, memory, UART, timers. Our kernel runs on it identically to real hardware.

Install:

- Windows: MSYS2 (pacman -S mingw-w64-x86_64-qemu-system-arm) or download from qemu.org
- Ubuntu: `sudo apt install qemu-system-arm`
- macOS: `brew install qemu`

Key QEMU flags:

- M lm3s6965evb Select the board
- nographic UART0 goes to terminal (no GUI window)
- kernel nexus-rtos.elf Load our ELF binary
- S Start paused (for GDB)
- gdb tcp::3333 Open GDB server on port 3333

To quit QEMU: press Ctrl+A, then X.

GDB -- The Debugger

arm-none-eabi-gdb comes with the compiler toolchain. We use it to:

- Step through code instruction by instruction
- Inspect registers, memory, and task stacks
- Set breakpoints on context switches and ISRs
- Watch variables like `current_task` in real time

The `.gdbinit` file auto-connects to QEMU and sets initial breakpoints.

4. Project Structure Walkthrough

The project is organized to separate concerns clearly:

include/ -- All header files. Each kernel component has one.
src/ -- All implementation files.
scripts/ -- Convenience scripts for running and debugging.
notes/ -- This PDF and the script that generates it.

Key files and what they do:

os_config.h

Tunable constants: MAX_TASKS, STACK_SIZE, HEAP_SIZE, clock speeds. Change these to experiment.

cortex_m3.h

Hardware register addresses (SysTick, SCB, UART, NVIC) and inline assembly intrinsics (__disable_irq, __set_PSP, etc).

startup.c

Vector table and Reset_Handler. Copies .data from flash to RAM, zeros .bss, calls main().

linker.ld

Tells the linker where to put code (flash) and data (SRAM). Defines _estack, _sdata, _sbss, etc.

context_switch.s

The heart of the RTOS: PendSV_Handler in ARM Thumb assembly. Saves/restores task contexts.

scheduler.c

Kernel core: ready queues (one per priority level), os_schedule(), os_init(), os_start().

task.c

Task creation (stack initialization), deletion, delay, yield.

sysclock.c

SysTick_Handler ISR: increments tick counter, wakes delayed tasks, triggers PendSV.

mutex.c

Mutex with priority inheritance protocol.

semaphore.c

Counting semaphore with blocking wait/post.

msgqueue.c

Fixed-size ring-buffer message queue with blocking send/receive.

heap.c

First-fit heap allocator with block splitting and forward coalescing.

uart.c

UART0 driver for the LM3S6965: init, putc, getc, puts, printf.

shell.c

Interactive debug shell: ps, top, free, uptime, trace, fault, crash, version, help.

ktrace.c

Lockless ring-buffer kernel event tracer. Logs context switches, mutex ops, faults.

fault.c

Stack canary checking, HardFault C handler, auto-restart of crashed tasks.

main.c

Demo application with producer/consumer, stats, shell, and stress-test tasks.

5. Startup Code & Vector Table

When the Cortex-M3 comes out of reset, it does exactly two things:

1. Loads the Main Stack Pointer (MSP) from address 0x00000000.
2. Branches to the address at 0x00000004 (the Reset_Handler).

Our startup.c provides both: a vector table in the .isr_vector section (placed at address 0 by the linker script), and the Reset_Handler function.

The Reset Handler

Reset_Handler does the C runtime setup that a normal OS would do for you:

1. Copy initialized global variables (.data section) from flash to SRAM. The linker stores them in flash (read-only), but the CPU expects them in RAM (read-write). The symbols _sidata, _sdata, _edata mark the source and destination.
2. Zero the .bss section (uninitialized globals). C guarantees these start at zero. The symbols _sbss, _ebss mark the range.
3. Call main(). If main returns, we spin in an infinite loop.

This is the ENTIRE C runtime for our RTOS. No crt0.o, no __libc_init.

Weak Aliases

Each exception handler is declared as a 'weak alias' of Default_Handler. This means: if we define PendSV_Handler elsewhere (we do, in assembly), the linker uses our version. If we don't define one (like DebugMon_Handler), it falls back to Default_Handler which just spins. This is a clean pattern for handling interrupts you haven't implemented yet.

6. The Linker Script

The linker script (linker.ld) tells the linker WHERE to place each section of our program in the target's memory. This is critical in embedded because there is no OS to handle memory mapping.

Our memory map:

```
MEMORY {  
    FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 256K  
    SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 64K  
}
```

_estack = top of SRAM (0x20010000). This is the initial MSP value.

Section placement:

- .isr_vector -> FLASH (must be at address 0x00000000)
- .text (code) -> FLASH
- .rodata (const data) -> FLASH
- .data (initialized globals) -> SRAM, but loaded from FLASH
The 'AT> FLASH' directive stores .data in flash; Reset_Handler copies it.
- .bss (uninitialized globals) -> SRAM
- ._heap (our heap pool) -> SRAM

The symbols (_sidata, _sdata, _edata, _sbss, _ebss) are generated by the linker and used by startup.c to perform the copy and zero operations.

7. Context Switching Deep Dive

Context switching is the CORE mechanism of any RTOS. It saves one task's CPU state and restores another's, giving the illusion that multiple tasks run simultaneously on a single core.

What is a Task's Context?

A task's context is the complete set of CPU registers that define its execution state:

- R0-R12: general-purpose registers
- SP (R13): stack pointer -- where the task's data lives
- LR (R14): link register -- where to return after a function call
- PC (R15): program counter -- the next instruction to execute
- xPSR: program status register (flags, thumb state, etc.)

To switch tasks, we save all of these to the current task's stack, then load all of them from the next task's stack.

How Cortex-M3 Helps Us

The Cortex-M3 hardware does HALF the work for us. When an exception fires:

Hardware automatically saves to the current stack:

R0, R1, R2, R3, R12, LR, PC, xPSR (8 registers)

We manually save in our PendSV handler:

R4, R5, R6, R7, R8, R9, R10, R11 (8 registers)

On exception return, hardware automatically restores the first 8. We restore R4-R11 manually before returning.

The PendSV Handler (context_switch.s)

PendSV (Pendable Supervisor Call) is an exception we trigger by software. We set it to the LOWEST priority so it runs only after all other ISRs are done. This is critical -- a context switch during a higher-priority ISR would corrupt state.

Step by step:

```
PendSV_Handler:
    CPSID I                ; Disable interrupts
    MRS  R0, PSP            ; R0 = current task's stack pointer
    CBZ  R0, skip_save     ; If PSP=0, this is the first switch

    ; Save R4-R11 onto the current task's stack
    STMDB R0!, {R4-R11}    ; Push R4-R11, decrement R0

    ; Save updated stack pointer to TCB
    LDR  R1, =current_task
    LDR  R1, [R1]           ; R1 = pointer to current TCB
    STR  R0, [R1, #0]       ; TCB.stack_ptr = R0
```

```

skip_save:
    BL  os_schedule          ; C function picks next task

    ; Load next task's stack pointer from TCB
    LDR R1, =current_task
    LDR R1, [R1]             ; R1 = pointer to next TCB
    LDR R0, [R1, #0]         ; R0 = next TCB.stack_ptr

    ; Restore R4-R11 from next task's stack
    LDMIA R0!, {R4-R11}     ; Pop R4-R11, increment R0

    MSR PSP, R0             ; Set PSP to next task's stack
    ORR LR, LR, #0x04       ; EXC_RETURN bit 2 = use PSP
    CPSIE I                 ; Re-enable interrupts
    BX  LR                  ; Exception return -> next task

```

Stack Frame Layout

After context save, a task's stack looks like this (low addr at top):

```

stack_ptr --> R4
             R5
             R6
             R7
             R8
             R9
             R10
             R11
PSP -----> R0  (hardware-saved frame)
             R1
             R2
             R3
             R12
             LR
             PC  (return address)
             xPSR

```

When we create a new task, we pre-fill this exact layout so the first context switch loads a valid frame and branches to the task function.

8. The Scheduler

Priority-Based Preemptive Scheduling

Our scheduler uses priority-based preemptive scheduling with round-robin within the same priority level.

How it works:

1. There are MAX_PRIORITY_LEVELS (8) priority levels, 0 (lowest) to 7 (highest).
2. Each level has a FIFO linked list of ready tasks (the 'ready queue').
3. os_schedule() scans from the highest priority down and picks the first task from the first non-empty queue.
4. Within the same priority, tasks rotate in FIFO order (round-robin).

'Preemptive' means: if a high-priority task becomes ready (e.g., its delay expires), it immediately preempts a lower-priority running task. The SysTick ISR checks this every 1 ms.

Ready Queue Operations

os_ready_queue_add(task): appends the task to the tail of its priority's list.

os_ready_queue_remove(task): removes the task from its priority's list.

The remove function uses the pointer-to-pointer pattern for clean singly-linked-list removal without special-casing the head.

os_schedule() Logic

```
void os_schedule(void) {
    // If current task was preempted (still RUNNING),
    // put it back in the ready queue
    if (current_task && current_task->state == RUNNING) {
        current_task->state = READY;
        os_ready_queue_add(current_task);
    }

    // Scan from highest to lowest priority
    for (i = MAX_PRIORITY-1; i >= 0; i--) {
        if (ready_queue[i] != NULL) {
            current_task = ready_queue[i];
            ready_queue[i] = current_task->next;
            current_task->state = RUNNING;
            return;
        }
    }
    // idle task at priority 0 guarantees this never fails
}
```

The Idle Task

The idle task runs at priority 0 and is always in the ready queue. It executes the WFI (Wait For Interrupt) instruction,

which puts the CPU into a low-power sleep until the next interrupt. This prevents the scheduler from ever having an empty queue.

9. Task Management

Task Control Block (TCB)

Each task is described by a TCB (Task Control Block) struct:

- `stack_ptr`: saved stack pointer (MUST be the first field -- assembly depends on this)
- `stack[STACK_SIZE]`: the task's private stack (2048 bytes by default)
- `name`: human-readable name for the debug shell
- `priority / base_priority`: current and original priority (for inheritance)
- `state`: READY, RUNNING, BLOCKED, SUSPENDED, or TERMINATED
- `delay_ticks`: countdown for `os_task_delay()`
- `id`: index into the `task_pool` array
- `next`: linked-list pointer for ready/wait queues

Creating a Task

`os_task_create()` does the following:

1. Finds a free slot in `task_pool` (`state == TERMINATED`).
2. Initializes the TCB fields (`name`, `priority`, `state = READY`).
3. Sets up the initial stack frame so it looks like the task was interrupted mid-execution. The PC slot points to the task function, R0 holds the argument, and LR points to a cleanup handler.
4. Adds the task to the appropriate ready queue.
5. Returns the task ID (slot index).

Stack Initialization

The key insight: we pre-fill the stack to mimic what the hardware would save during an exception. When `PendSV` restores this fake frame, the CPU thinks it is returning from an interrupt INTO the task function.

The `xPSR` is set to `0x01000000` (Thumb bit set -- Cortex-M3 ONLY runs Thumb code). The LR slot in the frame points to `task_return_handler()`, which calls `os_task_delete()` if a task's function returns.

Delays and Yielding

`os_task_delay(ticks)`: Sets `delay_ticks` and changes state to BLOCKED. The `SysTick` ISR decrements this each tick and wakes the task when it reaches zero.

`os_task_yield()`: Simply triggers `PendSV`. The current task stays RUNNING, so `os_schedule()` puts it back in the ready queue and picks the next one. This allows round-robin among equal-priority tasks.

10. SysTick Timer & System Clock

The SysTick timer is a 24-bit countdown timer built into every Cortex-M3. We configure it to fire every 1 ms (1000 Hz) as the RTOS heartbeat.

Configuration

```
void systick_init(void) {
    SYST_RVR = (50000000 / 1000) - 1; // 49999
    SYST_CVR = 0;
    SYST_CSR = CLKSOURCE | TICKINT | ENABLE;
}
```

At 50 MHz, the counter counts down from 49999 to 0, then reloads and fires the SysTick exception. 50000 clock cycles = 1 ms.

SysTick Handler

Every tick (1 ms), the handler:

1. Increments the global tick counter.
2. Scans all tasks: if BLOCKED with delay_ticks > 0, decrement it. When it hits 0, set state to READY and add to the ready queue.
3. Triggers PendSV to potentially switch to a newly-woken higher-priority task.

This is why PendSV must be the LOWEST priority exception -- it runs after SysTick finishes updating the ready queues.

11. Mutexes & Priority Inheritance

The Problem: Shared Resources

When two tasks access the same resource (like UART), they can corrupt each other's output. A mutex (mutual exclusion) ensures only one task holds the resource at a time.

Priority Inversion

Without priority inheritance, a dangerous scenario called 'priority inversion' can occur:

1. Low-priority task L acquires mutex M.
2. High-priority task H tries to acquire M, blocks.
3. Medium-priority task M (no relation to the mutex) preempts L.
4. H is now blocked by M, even though H has higher priority!

The Mars Pathfinder spacecraft experienced this exact bug in 1997, causing repeated system resets.

Our Solution: Priority Inheritance

When task H blocks on a mutex held by task L:

1. L's priority is temporarily boosted to H's priority.
2. L runs at the boosted priority, finishes its critical section faster.
3. When L releases the mutex, its priority reverts to base_priority.
4. H acquires the mutex and runs.

This prevents medium-priority tasks from preempting L while H is waiting.

Implementation

`mutex_lock()`:

- If unlocked: set lock=1, owner=current_task. Done.
- If locked: boost owner's priority if needed, block current task, add to wait_queue, trigger PendSV.

`mutex_unlock()`:

- Restore owner's base_priority.
- If wait_queue is non-empty: transfer ownership to the first waiter, wake it up. Otherwise: set lock=0, owner=NULL.

12. Semaphores

A semaphore is a signaling mechanism with a counter. Unlike a mutex (which is binary: locked/unlocked), a counting semaphore allows up to N concurrent accesses.

Use cases:

- Limiting access to a pool of N identical resources.
- Producer/consumer synchronization (producer posts, consumer waits).
- Event notification (one task signals another).

API

`sem_init(s, initial_count)`: Create with an initial count.

`sem_wait(s)`: If count > 0, decrement and continue. If count == 0, block.

`sem_post(s)`: If tasks are waiting, wake one. Otherwise increment count.

Key difference from mutex: a semaphore has no owner. Any task can post, and there is no priority inheritance. Use mutexes for mutual exclusion, semaphores for signaling/counting.

13. Message Queues (IPC)

Message queues allow tasks to exchange fixed-size messages through a shared buffer. They provide inter-process communication (IPC) with decoupling -- the producer and consumer don't need to run at the same time.

Ring Buffer Design

Our message queue uses a circular (ring) buffer:

- `buffer[]`: array of `MQ_MAX_MSGS` slots, each `MQ_MSG_SIZE` bytes.
- `head`: index of the next message to READ.
- `tail`: index of the next slot to WRITE.
- `count`: number of messages currently in the queue.

When `tail` reaches the end, it wraps to 0. Same for `head`. This gives $O(1)$ enqueue and dequeue with no shifting.

Blocking Behavior

`mq_send()`: If the queue is full, the sender blocks (added to `send_wait` queue). When a receiver takes a message, it wakes a blocked sender.

`mq_rcv()`: If the queue is empty, the receiver blocks (added to `rcv_wait` queue). When a sender adds a message, it wakes a blocked receiver.

This is the classic bounded producer-consumer pattern. Our demo uses it to show the producer sending messages that the consumer processes.

14. Heap Allocator

Our RTOS provides `os_malloc()` and `os_free()` for dynamic memory allocation. We implement a first-fit allocator with block splitting and forward coalescing.

How It Works

The heap is a statically-allocated array (16 KB by default). It is divided into blocks, each preceded by a header:

```
struct block { size, used, *next };
```

`os_malloc(size)`:

1. Walk the linked list of blocks.
2. Find the first FREE block large enough (first-fit).
3. If the block is much larger than needed, split it: carve off the requested size and create a new free block with the remainder.
4. Mark the block as used and return a pointer past the header.

`os_free(ptr)`:

1. Back up to the block header (`ptr - sizeof(header)`).
2. Mark the block as free.
3. If the NEXT block is also free, merge them (coalesce). This reduces fragmentation.

Limitations

- First-fit can cause fragmentation over time.
- We only coalesce forward (with the next block), not backward.
- No alignment beyond 4 bytes.
- Not thread-safe in the general case (we use `__disable_irq` as a crude lock).

For a production RTOS, you would use a more sophisticated allocator like TLSF (Two-Level Segregated Fit) which gives $O(1)$ allocation.

15. UART Driver & printf

The UART (Universal Asynchronous Receiver/Transmitter) is our only I/O channel. QEMU maps UART0 to the terminal, so `uart_putc('A')` makes 'A' appear in your console.

LM3S6965 UART Registers

UART0 base address: 0x4000C000

UARTDR (+0x000): Data Register -- write a byte to transmit, read to receive.

UARTFR (+0x018): Flag Register --

Bit 4 (RXFE): 1 = receive FIFO empty (nothing to read)

Bit 5 (TXFF): 1 = transmit FIFO full (can't write yet)

UARTIBRD (+0x024): Integer baud rate divisor

UARTFBRD (+0x028): Fractional baud rate divisor

UARTLCRH (+0x02C): Line control (word length, parity, FIFOs)

UARTCTL (+0x030): Control (enable UART, TX, RX)

Baud Rate Calculation

For 115200 baud at 50 MHz:

Divisor = $50000000 / (16 * 115200) = 27.1267$

Integer part (IBRD) = 27

Fractional part (FBRD) = $\text{round}(0.1267 * 64) = 8$

On QEMU, the baud rate doesn't actually matter (it's virtual), but setting it correctly is good practice and makes the code portable to real hardware.

Minimal printf

We implement `uart_printf()` with support for:

%d -- signed integer

%u -- unsigned integer

%x -- hexadecimal (with 0x prefix)

%s -- string

%c -- character

%% -- literal percent

This uses `<stdarg.h>` (`va_list`) which is provided by the compiler, not the C library, so it works in our freestanding environment.

16. Debug Shell

The shell runs as a regular task (priority 4 -- highest user priority) and provides an interactive command-line interface over UART. It reads characters one by one, builds a command line, and executes it when Enter is pressed.

Commands

`ps` -- Lists all active tasks with ID, name, priority, state, stack usage, and fault count.

`top` -- Per-task CPU usage: shows percentage of total ticks each task has consumed and context-switch counts.

`free` -- Shows heap memory usage: bytes used, bytes free, total pool size.

`uptime` -- Displays how long the system has been running (minutes, seconds, raw tick count).

`trace` -- Dumps the last 32 events from the KTrace ring buffer with timestamps, event types, and task IDs.

`fault` -- Shows the fault history log: every stack overflow and HardFault with timestamp, faulting PC/LR, and task ID.

`crash` -- Triggers a deliberate stack overflow on the 'stress' task to demonstrate the auto-recovery system.

`version` -- Kernel version and target info.

`help` -- Prints the command list.

Stack Usage Estimation

The 'ps' command estimates stack usage by counting how many words at the bottom of the stack array are still at their initial value. Since the stack grows downward from the top of the array, untouched words at the bottom indicate unused space. The canary words at the very bottom are excluded from this count.

17. Building & Running

Build

```
make          # Compile everything, produce nexus-rtos.elf and nexus-rtos.bin
```

What happens during the build:

1. Each .c file is compiled with `arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb`.
-ffreestanding: no standard library assumptions.
-nostdlib: don't link libc.
2. `context_switch.s` is assembled with `arm-none-eabi-gcc`.
3. All .o files are linked with our linker.ld script.
4. `objcopy` extracts a raw binary (`nexus-rtos.bin`) from the ELF.
5. `size` prints the final memory usage.

Run on QEMU

```
make run
# or directly:
qemu-system-arm -M lm3s6965evb -nographic -kernel nexus-rtos.elf
```

QEMU loads the ELF, sets up the emulated LM3S6965, and starts executing from the `Reset_Handler`. UART0 output appears in your terminal. You can type commands to the shell.

To quit: press Ctrl+A, then X.

Windows Notes

On Windows, the recommended setup is MSYS2 with MinGW64. Open an MSYS2 MinGW64 terminal and install the tools:

```
pacman -S mingw-w64-x86_64-arm-none-eabi-gcc
pacman -S mingw-w64-x86_64-qemu-system-arm
pacman -S make
```

Then just 'make' and 'make run' from the project directory.

Alternatively, you can use WSL (Windows Subsystem for Linux) with Ubuntu and follow the Ubuntu installation instructions. WSL works seamlessly.

18. Debugging with GDB

Starting a Debug Session

```
# Terminal 1: Start QEMU paused
make debug

# Terminal 2: Connect GDB
arm-none-eabi-gdb nexus-rtos.elf -x .gdbinit
```

The `.gdbinit` script automatically connects to QEMU on port 3333, loads the binary, and sets breakpoints on `main` and `HardFault_Handler`.

Essential GDB Commands

Navigation:

```
continue (c)    -- run until next breakpoint
step (s)        -- step into functions
next (n)        -- step over functions
finish          -- run until current function returns
```

Breakpoints:

```
break SysTick_Handler -- break on every tick
break PendSV_Handler  -- break on every context switch
break mutex_lock       -- break when any task acquires a mutex
```

Inspection:

```
print current_task->name -- which task is running?
print current_task->priority -- its priority level
print task_pool[0]       -- inspect task 0's full TCB
x/16x current_task->stack_ptr -- hex dump the stack
info registers           -- all CPU registers
```

Watchpoints:

```
watch current_task -- break whenever the running task changes
```

19. KTrace: Kernel Event Tracer

KTrace is a lightweight kernel event tracer that records what the kernel does in real time. Production RTOS kernels like FreeRTOS (with Tracealyzer) and Zephyr (with its tracing subsystem) have similar features. Building one from scratch demonstrates observability design.

Design Goals

1. Zero-allocation: uses a statically-allocated ring buffer.
2. Lockless: safe to call from ISR context without disabling interrupts (the ring buffer head is advanced atomically for a single producer).
3. Low overhead: each event is 8 bytes (timestamp, type, task_id, extra).
4. Configurable: TRACE_BUF_SIZE in os_config.h controls how many events are retained (default 256).

Event Types

EVT_TASK_SWITCH -- Context switch: which task is now running
 EVT_TASK_CREATE -- New task created: extra = priority
 EVT_TASK_DELETE -- Task terminated
 EVT_TASK_DELAY -- Task called os_task_delay(): extra = ticks
 EVT_TASK_WAKE -- Task woken from delay
 EVT_MUTEX_LOCK -- Mutex acquired
 EVT_MUTEX_UNLOCK -- Mutex released
 EVT_MUTEX_BLOCK -- Task blocked on mutex: extra = owner task ID
 EVT_FAULT -- HardFault occurred: extra = low 16 bits of faulting PC
 EVT_STACK_OVF -- Stack overflow detected
 EVT_TASK_RESTART -- Faulted task auto-restarted: extra = crash count

Ring Buffer Implementation

```
typedef struct {
    uint32_t timestamp;
    uint8_t  type;
    uint8_t  task_id;
    uint16_t extra;
} trace_event_t; // 8 bytes

static trace_event_t trace_buf[TRACE_BUF_SIZE];
static volatile uint32_t trace_head;

void ktrace_log(uint8_t type, uint8_t task_id, uint16_t extra) {
    trace_event_t *evt = &trace_buf[trace_head];
    evt->timestamp = os_get_ticks();
    evt->type      = type;
    evt->task_id   = task_id;
    evt->extra     = extra;
    trace_head = (trace_head + 1) % TRACE_BUF_SIZE;
}
```

How to Use

The trace is automatically populated by the kernel. Use the shell 'trace' command to dump the last 32 events. This is invaluable for debugging:

- See the exact sequence of context switches
- Find which task was running when a fault occurred
- Diagnose priority inversion by checking mutex block/unlock patterns
- Measure scheduling latency by comparing event timestamps

20. Fault Isolation & Auto-Recovery

Most teaching RTOS projects crash on any fault. NexusRTOS implements a fault isolation pipeline that detects, diagnoses, and recovers from task-level faults -- a key concept in safety-critical systems.

Stack Canary Painting

On task creation, the bottom 4 words of each task's stack are painted with the canary value 0xDEADBEEF. Since the ARM Cortex-M stack grows downward (from high addresses to low), a stack overflow will overwrite these canary words first.

This is the same technique used by GCC's -fstack-protector and production RTOS kernels.

```
// In os_task_create():
for (int i = 0; i < STACK_CANARY_WORDS; i++)
    tcb->stack[i] = STACK_CANARY; // 0xDEADBEEF
```

Round-Robin Canary Scanning

The SysTick ISR checks ONE task's canaries per tick, cycling through all tasks. With 16 task slots at 1000 Hz, every task is checked every 16 ms.

This amortized approach adds near-zero overhead per tick (checking 4 words), rather than scanning all tasks every tick.

HardFault Handler

For faults the canary scan cannot catch (e.g., dereferencing a bad pointer), the HardFault handler extracts the exception frame and calls a C handler:

1. Assembly wrapper determines whether MSP or PSP was in use (checks bit 2 of EXC_RETURN in LR).
2. Passes the stacked frame pointer and EXC_RETURN to HardFault_Handler_C.
3. The C handler logs diagnostics: faulting PC, LR, task name, task ID.
4. Records the fault in the fault log and KTrace.
5. If the task is marked 'restartable', rebuilds its stack frame and re-adds it to the ready queue.

```
HardFault_Handler:      ; in context_switch.s
    TST    LR, #4        ; Check EXC_RETURN bit 2
    ITE    EQ
    MRSEQ  R0, MSP        ; If 0: handler was using MSP
    MRSNE  R0, PSP        ; If 1: thread was using PSP
    MOV    R1, LR         ; Pass EXC_RETURN
    B      HardFault_Handler_C ; Tail-call to C handler
```

Auto-Recovery

Tasks created with .restartable = 1 are automatically restarted after a fault:

1. The TCB saves the original entry function and argument.

2. On fault, the stack is re-initialized to a fresh exception frame pointing to the original entry function.
3. Stack canaries are re-painted.
4. The task is added back to the ready queue.
5. A `fault_count` is incremented for monitoring.

The 'crash' shell command triggers a deliberate stack overflow on the 'stress' task to demonstrate this. The 'fault' command shows the fault history.

21. Per-Task CPU Profiler

The CPU profiler tracks how much execution time each task consumes. This is essential for performance analysis in real-time systems where meeting deadlines depends on understanding CPU budgets.

Implementation

The scheduler records timestamps on every context switch:

1. When a task is switched OUT, its `run_ticks` is incremented by $(\text{current_tick} - \text{last_switched_in})$.
2. When a task is switched IN, `last_switched_in` is set to the current tick.
3. `switch_count` is incremented each time a task is selected.

This gives us:

- CPU%: $(\text{run_ticks} / \text{uptime_ticks}) * 100$ for each task
- Switch frequency: how often the task is scheduled

The 'top' shell command displays this in a table.

What You Can Learn From It

- Which tasks dominate CPU time (is the idle task getting enough cycles?)
- Whether a task is switching too frequently (high overhead)
- CPU utilization: if the idle task gets < 20%, the system is heavily loaded
- Whether priority assignments match actual workload importance

In an interview, being able to explain how you measured and optimized CPU usage in your RTOS shows genuine systems engineering maturity.

22. Extending the Project

Ideas to make this project even more impressive:

Tickless Idle Mode

Instead of waking every 1 ms, calculate when the next task will be ready and sleep until then. This drastically reduces power consumption.

MPU (Memory Protection Unit)

Use the Cortex-M3's MPU to isolate each task's stack. A stack overflow triggers a MemManage exception instead of silently corrupting another task.

Interrupt-Driven UART

Replace polling with RX/TX interrupts and ring buffers. This frees the CPU while waiting for characters.

Priority Queue with Deadline

Implement EDF (Earliest Deadline First) scheduling alongside fixed-priority.

Software Timers

Allow tasks to register callbacks that fire after a delay without creating a full task.

Mutex with Recursive Locking

Allow the same task to lock a mutex multiple times without deadlocking.

CPU Usage Tracking

Measure how much time each task spends running vs. idle. Display in the shell with a 'top' command.

Port to Real Hardware

Flash this onto a real STM32 (Cortex-M3/M4) board. The code is nearly identical -- just update the UART and clock registers.

23. Interview Questions & Answers

Having this project on your resume WILL generate interview questions. Here are the most common ones and how to answer them:

Q: Why did you use PendSV for context switching instead of SysTick?

A: PendSV can be set to the lowest exception priority, ensuring the context switch happens AFTER all other ISRs complete. If we switched in SysTick, a higher-priority interrupt arriving mid-switch would see inconsistent state. PendSV is 'pendable' -- we set a flag and it fires when nothing else is pending.

Q: Explain priority inheritance in your mutex implementation.

A: When a high-priority task H blocks on a mutex held by low-priority task L, I temporarily boost L's priority to H's level. This prevents priority inversion, where a medium-priority task M could preempt L and indirectly block H. When L releases the mutex, its priority reverts. This is the same protocol that was patched into Mars Pathfinder's VxWorks.

Q: How does your scheduler achieve O(1) task selection?

A: Each priority level has its own ready queue (linked list). The scheduler scans from the highest priority downward -- with 8 levels, this is a constant-time scan. Within a level, tasks are dequeued from the head (FIFO), giving round-robin. A bitmap optimization could make this truly O(1) regardless of the number of priority levels.

Q: What happens when a task's stack overflows?

A: NexusRTOS detects it through stack canaries -- four words of 0xDEADBEEF painted at the bottom of each stack. The SysTick ISR scans one task per tick in round-robin, so every task is checked within 16 ms. On detection, the fault handler logs the event, terminates the task, and if it is marked restartable, re-initializes its stack and re-adds it to the ready queue. HardFaults are also caught via an assembly wrapper that extracts the exception frame and delegates to a C handler for diagnostics.

Q: Why not use the standard C library?

A: The standard library assumes an underlying OS for malloc, printf, file I/O, etc. We ARE the OS, so those assumptions don't hold. We use -ffreestanding -nostdlib and provide our own printf (via UART) and malloc (our heap allocator). The only compiler-provided headers we use are stdint.h, stddef.h, and stdarg.h.

Q: How would you port this to real hardware?

A: The Cortex-M3 core is identical across vendors -- the context switch, scheduler, and synchronization code work unchanged. You'd update: (1) the linker script for the new chip's memory map, (2) the UART register addresses and clock setup, and (3) any vendor-specific peripheral initialization. I'd start with an STM32F103 (Blue Pill board, about \$2).

Q: How does your kernel event tracer work and why did you build it?

A: KTrace is a lockless ring buffer of 8-byte events that logs context switches, mutex operations, task lifecycle, and faults with timestamps. I built it because observability is critical in real-time systems -- when debugging a priority inversion or missed deadline, you need a trace of what the kernel did, not just what the application printed. It is modeled after FreeRTOS Tracealyzer and Zephyr's tracing subsystem.

Q: How does your auto-recovery mechanism work? Is it safe?

A: When a fault is detected, the handler kills the faulting task, rebuilds its stack frame pointing to the original entry function, re-paints the canaries, and adds it back to the ready queue. This is safe because each task has its own stack -- we do not touch other tasks' memory. The fault count tracks how many times a task has crashed, which could be used to implement a 'three strikes' policy. In a production system, you would combine this with the MPU for stronger isolation.

Q: How do you measure per-task CPU usage?

A: The scheduler timestamps each context switch. When a task is switched out, `run_ticks += (now - last_switched_in)`. The 'top' command divides each task's `run_ticks` by total uptime to get a percentage. This is the same approach used by Linux's `/proc/stat`, just at the RTOS scale.